

StockLens Credentials Documentation

Overview

This document explains all credentials and authentication mechanisms used in StockLens.

User Credentials (Client-Side)

Users create login credentials by registering with the following fields:

Registration Fields

- Username - 3-80 characters
- Email - Valid email format
- Password - Must meet strength requirements:
 - Minimum 8 characters
 - At least 1 uppercase letter
 - At least 1 digit
- First Name
- Last Name

Example Test Credentials

```
{  
  "username": "johndoe123",  
  "email": "john@example.com",  
  "password": "SecurePass123",  
  "first_name": "John",  
  "last_name": "Doe"  
}
```

Authentication Tokens (JWT-Based)

After successful login, users receive two JWT tokens:

Access Token

- Purpose: Used for authenticated API requests

- Expiration: 1 hour
- Usage: Added to Authorization: Bearer token header

Refresh Token

- Purpose: Used to obtain new access tokens when expired
- Expiration: 30 days
- Storage: Stored in localStorage
- Auto-Refresh: Automatically refreshed before expiry

Token Storage

Tokens are stored in browser localStorage:

```
localStorage.setItem;
```

```
localStorage.setItem;
```

Environment Variables

Create a .env file in the project root with these credentials:

Frontend Configuration

```
REACT_APP_API_URL=https://web-production-dbfb6.up.railway.app/api/auth
```

```
REACT_APP_BACKEND_URL=https://web-production-dbfb6.up.railway.app
```

Backend Configuration

```
FLASK_ENV=production
```

```
SECRET_KEY=your-secret-key-here
```

```
JWT_SECRET_KEY=your-jwt-secret-key-here
```

```
DATABASE_URL=postgresql://user:password@host/dbname
```

AI Integration

```
GEMINI_API_KEY=your-gemini-api-key-here
```

Security Features

Password Security

- Bcrypt Hashing - Passwords hashed before storage

- Strength Validation - Enforced requirements
- Change Password - Users can update passwords anytime
- No Plain Text Storage - Never stored or logged

Token Security

- JWT-Based - Stateless authentication
- Automatic Refresh - Tokens refreshed before expiry
- Short Expiration - Access tokens expire in 1 hour
- HTTP-Only Storage - Can be optionally stored securely

Session Security

- Session Tracking - IP address & User-Agent logged
- Logout - Invalidates tokens server-side
- Protected Routes - Frontend route protection
- API Interceptors - Auto-redirect on 401 errors

API Endpoints

Authentication Routes (`/api/auth/`)

1. Register User

POST `/api/auth/register`

Content-Type: application/json

Request Body:

```
{
  "username": "johndoe123",
  "email": "john@example.com",
  "password": "SecurePass123",
  "first_name": "John",
  "last_name": "Doe"
}
```

Response: 201 Created

```
{
  "message": "Registration successful",
  "user": { ... },
  "access_token": "eyJhbGc...",
  "refresh_token": "eyJhbGc..."
}
```

2. Login User

POST /api/auth/login

Content-Type: application/json

Request Body:

```
{  
  "email": "john@example.com",  
  "password": "SecurePass123"  
}
```

Response: 200 OK

```
{  
  "message": "Login successful",  
  "user": { ... },  
  "access_token": "eyJhbGc...",  
  "refresh_token": "eyJhbGc..."  
}
```

3. Get Current User

GET /api/auth/me

Authorization: Bearer access_token

Response: 200 OK

```
{  
  "id": 1,  
  "username": "johndoe123",  
  "email": "john@example.com",  
  "first_name": "John",  
  "last_name": "Doe",  
  "avatar_url": null,  
  "theme": "dark",  
  "notifications_enabled": true,  
  "is_active": true,  
  "created_at": "2026-03-31T10:00:00Z"  
}
```

4. Refresh Token

POST /api/auth/refresh

Authorization: Bearer refresh_token

Response: 200 OK

```
{  
  "access_token": "eyJhbGc..."  
}
```

5. Update Profile

PUT /api/auth/profile

Authorization: Bearer access_token

Content-Type: application/json

Request Body:

```
{
  "first_name": "John",
  "last_name": "Smith",
  "bio": "Investor and trader",
  "avatar_url": "https://...",
  "theme": "dark"
}
```

Response: 200 OK

```
{
  "message": "Profile updated",
  "user": { ... }
}
```

6. Change Password

POST /api/auth/change-password

Authorization: Bearer access_token

Content-Type: application/json

Request Body:

```
{
  "old_password": "OldPass123",
  "new_password": "NewPass456"
}
```

Response: 200 OK

```
{
  "message": "Password changed successfully"
}
```

7. Logout

POST /api/auth/logout

Authorization: Bearer access_token

Response: 200 OK

```
{
  "message": "Logout successful"
}
```

Database Models

User Model

- id - Primary key
- username - Unique username
- email - Unique email
- password_hash - Bcrypt hashed password
- first_name - First name
- last_name - Last name
- bio - User biography
- avatar_url - Profile picture URL
- theme - UI theme preference
- notifications_enabled - Global notification toggle
- email_alerts - Email alerts toggle
- is_active - Account status
- email_verified - Email verification status
- created_at - Account creation timestamp
- updated_at - Last update timestamp
- last_login - Last login timestamp

Related Models

- UserSession - Session tracking for security audits
- Watchlist - User's stock watchlist
- PortfolioHolding - Portfolio positions
- PriceAlert - User-created price alerts

Implementation Files

Backend Files

- backend/auth.py - Authentication routes
- backend/models.py - Database models
- backend/server.py - Flask app with JWT setup

Frontend Files

- src/pages/Login.js - Login page component
- src/pages/Register.js - Registration page component
- src/context/AuthContext.js - Auth state management
- src/components/ProtectedRoute.js - Protected route wrapper

- src/services/api.js - Axios interceptors for JWT

Quick Start

1. Register a New Account

1. Go to <http://localhost:3000/register>
2. Fill in registration form
3. Click "Register"
4. Redirected to home page

2. Login

1. Go to <http://localhost:3000/login>
2. Enter email and password
3. Click "Login"
4. Redirected to home page

3. Access Protected Pages

- /portfolio - Portfolio tracking
- /watchlist - Stock watchlist
- /analysis - Stock analysis
- /predictions - Price predictions
- /settings - User settings
- All require authentication

4. Logout

1. Click user menu/profile icon
2. Select "Logout"
3. Tokens cleared from localStorage
4. Redirected to login page

Security Best Practices

For Development

- Use .env.example as template
- Never commit .env file to Git

- Use strong SECRET_KEY and JWT_SECRET_KEY values
- Test password validation rules

For Production

- Use HTTPS only
- Set FLASK_ENV=production
- Use strong, randomly generated secret keys
- Enable CORS restrictions
- Implement rate limiting
- Monitor failed login attempts
- Use database backups
- Enable SSL/TLS certificates

Troubleshooting

Login Fails

- Check email/password credentials
- Ensure account is registered first
- Verify API URL is correct in .env
- Check backend is running

Tokens Expired

- App automatically refreshes tokens
- Required: Valid refresh_token in localStorage
- If refresh fails, logout and login again

Protected Routes Redirect to Login

- Check if access token is valid
- Verify token exists in localStorage
- Try refreshing the page
- Clear localStorage and login again

Support

For authentication issues, check:

1. AUTHENTICATION.md - Full auth documentation

2. backend/auth.py - Backend implementation

3. Browser DevTools Application LocalStorage - Token verification